

# FINAL SCRIPT PACKAGE

## Recommended Title

Claude Code 实操上手：4 个场景教你真的拿它干活

## Alternate Titles

1. 别再空聊了，Claude Code 正确干活流程
2. 我用 4 个真实任务，把 Claude Code 用法讲明白

## Thumbnail Copy

1. 直接开干
2. 4 个真实场景
3. 别只会聊天

## Positioning

这不是一条“Claude Code 有哪些功能”的介绍视频，而是一条实操教程。目标是让观众看完后能直接照着做，而不是只获得模糊印象。

## Audience

- 已经会写代码，但还没把 Claude Code 真正用进工作流的人
- 想用 AI 提速日常开发，而不是只想看演示的人
- 独立开发者、工程师、技术内容受众

## Runtime

14–18 分钟

# Core Promise

看完之后，观众能直接完成 4 类任务：

1. 安装并启动 Claude Code。
2. 用 Claude Code 快速读懂陌生项目。
3. 用 Claude Code 处理报错、修 bug、跑验证。
4. 用 CLAUDE.md、/memory、.claude/settings.json、/status 配出长期可用的协作环境。

## Recording Script

如果你现在对 Claude Code 的理解，还停留在“一个待在终端里的 AI 聊天工具”，那你大概率已经把它用偏了。

因为 Claude Code 最值钱的地方，从来不是陪你空聊，而是直接进项目、读代码、跑命令、改文件、验证结果。

所以这期我不讲抽象概念，不讲谁比谁强，也不做大而空的功能盘点。

这期我们只做一件事。

我用 4 个真实开发场景，带你把 Claude Code 真正用起来。

看完以后你至少应该能马上上手 4 件事：

第一，装好 Claude Code 并进到真实项目里。第二，用它快速看懂一个陌生代码库。第三，用它从报错一路走到修复和验证。第四，把记忆、规则和权限边界一次配好，避免后面越用越乱。

如果你要的是能干活的版本，这期就是给你的。

### 第一段：先装上，再进项目，别在空气里聊天

到 2026 年 3 月 23 日，Anthropic 官方 Quickstart 里给的 macOS、Linux、WSL 推荐安装方式是：

```
curl -fsSL https://claude.ai/install.sh | bash
```

如果你习惯 Homebrew，也可以：

```
brew install --cask claude-code
```

装完以后进入你的真实项目目录：

```
cd /path/to/project
claude
```

第一次运行时提示登录；如果之后要切换账号，可以在交互里输入：

```
/login
```

启动后第一轮不要问空话，直接问这 4 句：

```
give me an overview of this codebase
explain the main architecture patterns used here
what are the key data models?
where is the main entry point?
```

这一步的目标不是写代码，而是先建立地图。

## 第二段：场景一，接手陌生项目，10 分钟摸清核心路径

假设你接到一个登录相关需求，你先不要自己盲搜。

先问：

```
find the files that handle user authentication
```

再问：

```
how do these authentication files work together?
```

最后把整条链路串起来：

```
trace the login process from front-end to database
```

这一套动作特别适合刚接手代码库时用。Claude Code 最擅长的一类活，就是先替你“找入口、串流程、拉上下文”。

## 第三段：场景二，从报错到修复，不要一句“帮我修”

修 bug 的正确顺序应该固定成：

复现 -> 根因 -> 方案 -> 修改 -> 验证

可以直接这样开始：

```
I'm seeing an error when I run npm test.
Here is the full stack trace.
Please explain the root cause first, then give me 2-3 fix options.
Prefer the most conservative fix that keeps behavior unchanged.
```

官方 workflow 里的典型动作包括：

```
I'm seeing an error when I run npm test
suggest a few ways to fix the @ts-ignore in user.ts
update user.ts to add the null check you suggested
```

改完以后必须补验证：

```
run tests for the refactored code
```

不要一句“帮我修”，而是把修 bug 这件事拆成 Claude Code 擅长的几个阶段。

## 第四段：场景三，重构和批量工程活，才是它最容易省时间的地方

如果你要清工程债，可以直接按这个流程来：

```
find deprecated API usage in our codebase
suggest how to refactor utils.js to use modern JavaScript features
refactor utils.js to use ES2024 features while maintaining the same
behavior
run tests for the refactored code
```

其中最关键的一句是：

```
while maintaining the same behavior
```

你要把“行为不变”说死，Claude Code 才更适合帮你扫老代码，而不是顺手给你做一轮不必要的大改。

## 第五段：场景四，把规则和权限配好，不然早晚翻车

长期使用时请记住：

- CLAUDE.md 管项目规则
- settings.json 管权限和行为
- /memory 看当前加载了哪些记忆
- /status 看哪些设置层正在生效

一个非常实用的 CLAUDE.md 可以写成这样：

# CLAUDE.md

- use pnpm, not npm
- run unit tests before proposing a commit
- do not edit files under generated/
- prefer minimal diffs over broad rewrites

权限配置至少先挡住敏感文件：

```
{
  "permissions": {
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)"
    ]
  }
}
```

```
    ]  
  }  
}
```

如果你改了配置却发现行为没变，就直接跑：

```
/status
```

看清楚到底是哪一层配置在生效。

## 第六段：Bonus，把 Claude Code 当命令行工具塞进现有流程

如果你不想每次都开交互会话，可以直接用一次性命令：

```
cat build-error.txt | claude -p 'concisely explain the root cause of  
this build error' > output.txt  
cat code.py | claude -p 'analyze this code for bugs' --output-format  
json > analysis.json
```

这让 Claude Code 不只是一个 REPL，也能成为 shell 工具链里的一环。

## Ending Summary

真正高效的 Claude Code 用法，不是一直聊天，而是把它塞进真实项目、真实任务、真实验证流程里。

你只要记住 5 个动作：

1. 先进入项目，再启动 claude。
2. 第一次先问地图，不要急着改代码。
3. 修 bug 固定顺序：复现、根因、方案、修改、验证。
4. 重构时把“行为不变”说清楚。
5. 尽早配好 CLAUDE.md、/memory、权限和 /status。

## Recording Notes

- 画面不要一直停在 talking head，最好跟着终端实录切。
- 第二段、第三段、第四段都可以直接上命令和 prompt 的逐条演示。
- 这一版最适合配“边讲边做”的节奏，而不是纯口播。

## Source Links

- [Quickstart](#)
- [Common workflows](#)

- Memory
- Settings